

NetSage AI — how it is built

Every file, every language, and how they fit together

Submitted by	Danish Soreng
Institution	KIIT University
Application	NetSage AI — network fault triage with a human review step
Measured	Line counts below are taken from the source files, not estimated

1. Languages used, by weight

The project is written in six languages, though the distribution is lopsided on purpose. Python does all of the reasoning, storage and routing; JavaScript only handles what happens inside the browser window.

Language	Lines	Where it lives	What it does
Python 3.12	1,877	app/, scripts/, airflow/, run.py	All server-side work: the rule checker, the AI calls, the database, the routes
JavaScript	788	inside the two .dc.html files	Console behaviour — form state, calls to the API, the offline fallback
HTML / Jinja2	750	app/templates/, netsage-ui/	Page structure. 312 lines server-rendered, 438 lines of console markup
CSS	269	app/static/style.css	Styling for the server-rendered pages
JSON	302	data/cases/*.json	The 30-case evaluation set. Data, not code
SQL	55	schema.sql	PostgreSQL table definitions, kept for reference

One file is generated, not written. netsage-ui/support.js is 1,911 lines of Design Component runtime, and its own header says "GENERATED — do not edit". It is excluded from the counts above, and it should not be presented as authored work.

The headline is that **Python is roughly seventy per cent of the authored code**. This is a Python application with a JavaScript front end, not a JavaScript application.

2. Directory layout

```
netsage/
|-- run.py                <- entry point (71 lines)
|-- requirements.txt      dependencies
|-- .env.example          configuration template
|-- schema.sql            PostgreSQL DDL (reference)
|
|-- app/                  the application - 1,569 lines of Python
|   |-- config.py         settings, and the .env reader
|   |-- models.py         the four tables
|   |-- db.py             connection and session handling
|   |-- rule_checker.py   the eight deterministic checks
|   |-- ai_assistant.py   Claude, plus the offline engine
|   |-- pipeline.py       the four stages
|   |-- web.py            page routes
|   |-- api.py            the JSON API
|   |-- templates/       five Jinja2 pages
|   +-- static/style.css
|
|-- netsage-ui/           the console: two design files + runtime
|-- airflow/dags/         the scheduled pipeline
|-- scripts/              CLI utilities and the smoke test
|-- data/cases/           the 30 seed cases
+-- docs/                 reports
```

3. Every file, by layer

Layer 0 — entry point

File	Lines	Purpose
run.py	71	The only file that is run. Creates the tables, seeds the 30 cases on the first run only, prints the URLs, and starts the server on port 5000. The PyCharm run configuration points here.

Layer 1 — foundation

File	Lines	Purpose
app/config.py	67	Reads settings from the environment. Contains a hand-written .env parser rather than a third-party one, so nothing is hidden. Sets the database URL, the API key, the model, and the 0.55 low-confidence threshold.
app/models.py	136	The four tables defined as Python classes through SQLAlchemy's ORM. Because the schema is described in Python rather than SQL, the same code runs on SQLite and PostgreSQL unchanged.
app/db.py	39	The connection, and the session_scope() context manager that commits on success and rolls back on error. Every database operation goes through it.

Layer 2 — the logic, with no framework in it

File	Lines	Purpose
app/rule_checker.py	234	Pure Python. No AI, no network, no imports beyond ipaddress, re and collections. Eight functions, one per check, plus a regular-expression host parser. Identical input always produces identical findings.

File	Lines	Purpose
app/ai_assistant.py	379	The largest file. Holds the ProposedDiagnosis output schema, the Claude call through the official SDK, the offline heuristic engine, a normalisation step that coerces the reply into the shape the database expects, and chat() for the console assistant.
app/pipeline.py	204	The four stages as four ordinary functions. Both the web application and Airflow call these, which is what makes 'one implementation, two entry points' a fact rather than a claim.

Layer 3 — the web layer

File	Lines	Purpose
app/web.py	175	Maps URLs to pages: /, /overview, /queue, /case/<ref>, /dashboard, and /support.js.
app/api.py	335	The JSON API the console runs on: POST /api/triage, /api/review, /api/chat, and GET /api/metrics, /api/health. Also allocates the LIVE-#### case references.
app/templates/	312	Five Jinja2 files - base, index, case_detail (134 lines, the largest), dashboard, not_found. Jinja2 is HTML with {% %} loops and {{ }} values filled in by Python.
app/static/style.css	269	Styling for the server-rendered pages.

Layer 4 — the console

File	Lines	Purpose
NetSage Triage.dc.html	768	Self-contained: 237 lines of markup using {{ }} bindings and <sc-for> / <sc-if> tags, and 531 lines of JavaScript holding the form state, runDiagnosis(), sendChat(), recordReview(), the offline fallback checker and the three.js background.
NetSage Console.dc.html	458	The overview: 201 lines of markup and 257 of JavaScript, most of it loadMetrics() and renderVals().
support.js	1,911	GENERATED. The runtime that makes those two files work; it loads React and Babel from a CDN. Not authored as part of this project.

Layer 5 — orchestration and utilities

File	Lines	Purpose
airflow/dags/netsage_pipeline.py	103	The hourly DAG. Orchestration only - it contains no diagnosis logic, which is what keeps the two entry points honest.
scripts/smoke_test.py	109	Automated verification of all thirteen routes, run from an empty database.
scripts/run_pipeline.py	15	Runs the pipeline without Airflow.
scripts/init_db.py	10	Creates the tables and nothing else.

4. Dependencies

Four packages are declared in requirements.txt. Ten more arrive automatically as sub-dependencies.

Package	Version	Declared?	Why it is there
Flask	3.1.3	yes	The web framework. Turns Python functions into URLs
SQLAlchemy	2.0.52	yes	The ORM. Python classes become database tables
anthropic	1.2.0	yes	The official Claude API SDK
psycopg2-binary	2.9.12	yes	PostgreSQL driver. Only needed if you move off SQLite
Jinja2	3.1.6	no	The template engine. Arrives with Flask
pydantic	2.13.5	no	Arrives with anthropic. ProposedDiagnosis is built on it

SQLite requires no package at all - it is built into Python. That is why the whole system installs with three lines and runs with no database server, no Node.js, no build step and no compiler.

5. How the files connect

One press of *Run diagnosis* in the console, traced through the codebase. Every arrow below is a real import that can be followed in the source.

```

browser    (JavaScript inside NetSage Triage.dc.html)
  |  fetch POST /api/triage
  v
app/web.py          routes the URL
  v
app/api.py          triage()
  |--> app/rule_checker.py  check()      <- plain Python, no AI
  |--> app/ai_assistant.py  diagnose()   <- findings passed in as facts
  |      |--> anthropic SDK --> Claude API
  |      +--> or the local heuristic engine
  +--> app/db.py + app/models.py        <- writes three tables
  v
JSON returned to the browser; the page re-renders

```

6. Setting it up from nothing

```

python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env
python run.py

```

Five commands. No database server to install, no Node.js, no build step - the console ships as plain HTML the browser reads directly. The only external requirements are internet access for the console's CDN libraries, and an API key if Claude is wanted rather than the offline engine.

7. Points worth stating plainly

- **support.js is generated.** It is 1,911 lines and was not written for this project. Saying so is better than being asked.
- **The four largest authored files are all Python** - ai_assistant.py (379), api.py (335), rule_checker.py (234) and pipeline.py (204). That is where the project actually lives.
- **schema.sql is reference, not the source of truth.** The tables are defined in app/models.py; the SQL file documents what SQLAlchemy produces for PostgreSQL.
- **The rule checker imports nothing but the standard library.** No AI, no network, no third-party package - which is exactly why its findings can be called facts.